



Experiment 1A : Introduction to MATLAB: Basic commands, symbolic variables, arithmetic operations, and plotting of standard functions

1. Write an algorithm for Introduction to MATLAB (Basic commands, symbolic variables, arithmetic operations, and plotting of standard functions).

Algorithm for Introduction to MATLAB (Basic commands, symbolic variables, arithmetic operations, and plotting of standard functions): To familiarize students with the MATLAB environment, variables, basic commands, arithmetic operations, symbolic variables, built-in mathematical functions, and graphical visualization of standard mathematical functions.

Input: Numerical values, symbolic variables, and mathematical expressions.

Output: Numerical results, symbolic computations, and graphical plots.

Step 1: Launch MATLAB and open a new script file or notebook.

Step 2: Explore the development environment, including the Command Window/Console, Workspace or Variable Explorer, Current Folder, Editor, and Command History.

Step 3: Create variables and assign numerical values.

Step 4: Perform arithmetic operations such as addition, subtraction, multiplication, division, modulus, and exponentiation.

Step 5: Evaluate built-in mathematical functions such as square root, exponential, logarithmic, and trigonometric functions.

Step 6: Define symbolic variables and create symbolic mathematical expressions.

Step 7: Perform symbolic computations such as simplification, expansion, and factorization.

Step 8: Generate a suitable range of values for the independent variable.

Step 9: Plot a standard mathematical function using appropriate plotting commands.

Step 10: Add a title, axis labels, legend (if required), and grid to the graph.

Step 11: Display the numerical results, symbolic results, and graphical output.

Step 12: Save the program and terminate the execution.

2. Create two variables $a = 15$ and $b = 5$. Perform the following operations:

- Addition
- Subtraction
- Multiplication
- Division
- Exponentiation
- Modulus

3. Evaluate the following mathematical expressions: $\sqrt{64}$, e^2 , $\sin\left(\frac{\pi}{6}\right)$, $\cos\left(\frac{\pi}{3}\right)$, $\log(100)$.
4. Define the symbolic expression $x^2 + 5x + 6$. Using MATLAB, Factorize the given expression.
5. Plot the following functions over the interval $0 \leq x \leq 2\pi$.
 - $y = \sin x$
 - $y = \cos x$
 Include the graph title, axis labels, and grid.
6. A company's monthly profit is modeled by $P(x) = x^2 + 4x + 5$, where x represents the number of months. Using MATLAB,
 - Define the function.
 - Evaluate the profit for $x = 5$
 - Plot the function for $0 \leq x \leq 10$.

%% Experiment 1A: Introduction to MATLAB

% Aim:

**% To perform basic arithmetic operations, symbolic computations,
% and plot a standard mathematical function.**

clc;

clear;

close all;

%% Step 3: Create variables and assign numerical values

a = 15;

b = 4;

%% Step 4: Perform arithmetic operations

addition = a + b;

subtraction = a - b;

multiplication = a * b;

division = a / b;

modulus = mod(a,b);

powerValue = a^b;

%% Step 5: Evaluate built-in mathematical functions

squareRoot = sqrt(a);

exponential = exp(2);

logarithm = log(a); **% Natural logarithm**

sineValue = sin(pi/6);

```
cosineValue = cos(pi/3);  
tangentValue = tan(pi/4);
```

%% Step 6: Define symbolic variables and expressions

```
syms x
```

```
f = x^2 + 5*x + 6;
```

%% Step 7: Perform symbolic computations

```
simplifiedExpr = simplify((x^2 - 1)/(x - 1));  
expandedExpr = expand((x + 2)^3);  
factoredExpr = factor(f);
```

%% Step 8: Generate values for the independent variable

```
xValues = -5:0.1:5;
```

%% Step 9: Plot a standard mathematical function

```
yValues = xValues.^2;
```

```
figure;  
plot(xValues, yValues, 'b', 'LineWidth', 2);
```

%% Step 10: Add title, labels, legend, and grid

```
title('Graph of  $y = x^2$ ');  
xlabel('x');  
ylabel('y');  
legend('y = x^2', 'Location', 'northwest');  
grid on;
```

%% Step 11: Display numerical and symbolic results

```
disp('=====');  
disp('    Numerical Computations');  
disp('=====');
```

```

fprintf('Addition      = %.2f\n', addition);
fprintf('Subtraction   = %.2f\n', subtraction);
fprintf('Multiplication = %.2f\n', multiplication);
fprintf('Division      = %.2f\n', division);
fprintf('Modulus       = %.2f\n', modulus);
fprintf('Exponentiation (a^b) = %.2f\n', powerValue);

fprintf('\nSquare Root of a   = %.4f\n', squareRoot);
fprintf('Exponential exp(2)   = %.4f\n', exponential);
fprintf('Natural Log of a     = %.4f\n', logarithm);
fprintf('sin(pi/6)           = %.4f\n', sineValue);
fprintf('cos(pi/3)            = %.4f\n', cosineValue);
fprintf('tan(pi/4)            = %.4f\n', tangentValue);

disp(' ');
disp('=====');
disp('      Symbolic Computations');
disp('=====');

disp('Original Expression:');
disp(f);

disp('Simplified Expression:');
disp(simplifiedExpr);

disp('Expanded Expression:');
disp(expandedExpr);

disp('Factored Expression:');
disp(factoredExpr);

disp(' ');
disp('Program executed successfully.');
```



Experiment 1B : Introduction to MATLAB: Decision Making and Looping Statements in MATLAB

1. Write an algorithm for Introduction to MATLAB (Decision Making and Looping Statements in MATLAB).

Algorithm for Introduction to MATLAB (Decision Making and Looping Statements in MATLAB): To familiarize students with conditional statements and iterative programming using MATLAB.

Input: Numerical values and logical conditions.

Output: Results obtained using conditional statements and looping constructs.

Step 1: Launch MATLAB and create a new script file.

Step 2: Define the required variables.

Step 3: Read or assign input values.

Step 4: Implement conditional statements (if, if-else, nested if, or switch-case/match-case) to evaluate logical conditions.

Step 5: Implement a for loop to execute statements for a fixed number of iterations.

Step 6: Implement a while loop to execute statements until a specified condition becomes false.

Step 7: Use break and continue statements to control loop execution where required.

Step 8: Display the output.

Step 9: Save the program and terminate the execution.

2. Write a MATLAB program to determine whether a given number is positive, negative, or zero.

3. Write a MATLAB program to determine whether a given year is a leap year.

4. Write a MATLAB program to find the largest among three numbers using the **if-else** statement.

5. Using a **for** loop, compute the sum of the first 20 natural numbers.

6. Using a **while** loop, generate the multiplication table of a given number up to 10.

7. Using a **for** loop, generate and display the squares and cubes of the numbers from **1 to 10** in tabular form.

Solution:

```
%% Program 2: Check Positive, Negative or Zero
clc;
```